CMSC 180
Introduction to Parallel Computing
Second Semester AY 2025-2026

Laboratory Research Problem 02 (LRP02)
Runtime-efficient Threaded Min-Max Transformation of Elements in a Column of a Matrix

Introduction

Given an $m \times n$ matrix **X** with m rows and n columns, an $m \times n$ matrix **T** holds the Min-Max Transformation (MMT) of the elements in columns of **X** as in Equation 1 of the previous LRP01.

Research Question 1: What do you think is the complexity of solving the MMT of the columns in an $n \times n$ square matrix **X** when using n concurrent processes? The obvious process assignment is one column of **X** for each process.

Research Question 2: What do you think is the complexity of solving the MMT of the columns in an $n \times n$ square matrix **X** when using $n/2$ concurrent processes (what is the obvious process assignment here)? What about with $n/4$ concurrent processes (i.e., process assignment)? What about with $n/8$ concurrent processes? What about with n/m concurrent processes, where $n>>m$? Is the process assignment still obvious at n/m concurrent processes?

Laboratory Activity 1: Extend the serially efficient computer program that you wrote in LRP01 to use process threads to compute the MMTs of the elements in each of the columns of an $n \times n$ square matrix **X**. In other words, transform your efficient serial computer program into a (hopefully more efficient and faster) threaded computer program, where a thread is a lightweight process.

How to do it?

1. Write the main program `lab02` that includes the following:
   (1) Read n and t as user inputs (maybe from a command line or as a data stream), where n is the size of the square matrix, t is the number of threads to create, and $n >> t$;
   (2) Create a non-zero $n \times n$ square matrix **X** whose elements are assigned with random non-zero integers;
   (3) Divide your **X** into t submatrices of size $n \times n/t$ each, which we will respectively call as the submatrices $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_t$;
   (4) Take note of the system time `time_before`;
   (5) Create t threads, where in the ith thread call `mmt`($\boldsymbol{x}_i$, n, n/t) for all $1 \le i \le t$;← very important
   (6) Recreate the correct **T** from the output of each threaded `mmt()`;← very important
   (7) Take note of the system time `time_after`;
   (8) Obtain the elapsed time `time_elapsed:=time_after - time_before`;
   (9) Output `time_elapsed`;

2. Fill in the following table with your time readings:

n	*t*	Time Elapsed (seconds)			Average Runtime (seconds)
		Run 1	**Run 2**	**Run 3**	
25,000	1*				
25,000	2				
25,000	4				
25,000	8				
25,000	16				
25,000	32				
25,000	64				

*This should be closed but a little bit higher to the average that you obtained in LRP01.

Research Question 3: Why do you think that $t = 1$ will be a little bit higher than the average that was obtained in LRP01?

Research Question 4: In step (3) in number 1 above, explain what will happen if we divide **X** into $n/t \times n$ instead? How are we going to do it so that the same answer can be arrived at?

Laboratory Activity 2: With `lab02`, repeat the activities in LRP01 for $n = 30{,}000$ and $n = 40{,}000$. Do you think you can now achieve $n = 50{,}000$ and even $n = 100{,}000$? Try it to see if you can. If you were able to do so, why do you think you can now do it? If not yet, why do you think you still can not?

3. Using a graphing software for each *n*, graph *t* versus **Average** obtained from the Table above. Describe in detail what you have observed. Do you think you can go as far as $t = n$? If not, what about $t = n/2$? Or, $t = n/4$? Or, $t = n/8$?

Laboratory Activity 3: Repeat laboratory activity 1 but for the division of **X** as described in Research Question 4. What sort of thing do you need to do so that this division can be implemented? Graph the average as in number 3 above, if obtained.

Solutions to Possible Problems that may Come up

Problem 1 : *I do not know how to write threaded programs*.
Answer 1 : *Is that really a problem?*

Problem 2 : *The programming language I used for LRP01 does not have a robust library for writing threaded codes.*
Answer 2 : *See Answer 1.*

Problem 3 : *My* [*boy*|*girl*]*friend broke up with me today and I can not really concentrate on this exercise.*
Answer 3 : *See Answer 1.*

Problem 4 : *I have not eaten any meal since last meeting.*
Answer 4 : *Pretend you are a freshman and attend any org's orientation.*

Some Helpful Online Resources (also listed in Google Classroom)

1. For Java: https://beginnersbook.com/2013/03/multithreading-in-java
2. For C: https://www.geeksforgeeks.org/multithreading-c-2
3. For Python: https://realpython.com/intro-to-python-threading
4. For Perl: https://metacpan.org/pod/distribution/perl/pod/perlthrtut.pod

If our favorite PL is not listed here, we must consider adding another PL as our favorite.